



UNIVERSITÉ EURO-MÉDITERRANÉENNE DE FÈS
École d'Ingénierie Digitale et d'Intelligence Artificielle (EIDIA)

Rapport PFA

Data Lake Analytics avec Azure Synapse et Power BI

Étudiant : Yassir Tagemouati (Tage Yassir)

Filière : Cycle Ingénieur — Big Data

Commanditaire : Smartovate Ltd

Encadrant Académique : Mme Loubna Ourabah (EIDIA, UEMF)

Encadrant Professionnel : M. Abdelkhalek Bakkari (Fondateur & CEO, Smartovate)

Année Universitaire : 2025 – 2026

Table des matières

Remerciements	4
Résumé	5
Introduction Générale	6
Partie 1 — Présentation de l'organisation	8
1.1 Smartovate : présentation de l'entreprise	8
1.2 Mission pédagogique	9
1.3 Rôle de l'étudiant	9
Partie 2 — Analyse des Besoins Fonctionnels	11
2.1 Introduction	11
2.2 Périmètre et Contexte	11
2.3 Sources de données (Oracle SCM simulé)	11
2.4 Exigences fonctionnelles détaillées	12
2.5 Exigences non-fonctionnelles	13
2.6 Règles de qualité des données (Data Quality)	13
2.7 Mapping Bronze / Silver / Gold	13
2.8 Flux d'orchestration et contrôles	14
2.9 SLA et critères d'acceptation	14
2.10 Histoires utilisateurs (User Stories)	15
2.11 Traçabilité et journaux	15
2.12 Conclusion de l'analyse	15
Partie 3 — Conception de l'Architecture et Choix Techniques	17
3.1 Introduction	17
3.2 Vue d'architecture logique	17
3.3 Architecture physique et ressources Azure	19
3.4 Organisation du Data Lake (Structure Medallion)	20
3.5 Design des pipelines ADF	23
3.6 Traitements et transformations (Synapse PySpark)	24

3.7	Schéma Gold — Modélisation pour BI	25
3.8	Conception des Dimensions et Modélisation Gold	26
3.9	Sécurité et gestion des accès	27
3.10	Monitoring, logging et alerting	27
3.11	Opérations et runbook (exploitation)	27
3.12	CI/CD et Infrastructure as Code (recommandations)	28
3.13	Performances, dimensionnement et coûts	28
3.14	Annexes et références dans le dépôt	29
3.15	Conclusion	29
Partie 4 — Implémentation de la Plateforme		31
4.1	Introduction	31
4.2	Environnements et prérequis	31
4.3	Génération des données (Oracle SCM simulé)	31
4.4	Téléversement initial vers ADLS (AutomaticUpload.py)	32
4.5	Contrôle d'ingestion (ingestion_status)	32
4.6	Orchestration : Azure Data Factory (ADF)	33
4.6.1	MasterIngestioDaily (pipeline maître)	33
4.6.2	BronzeToSilver (pipeline enfant)	33
4.6.3	MasterSilverToGold (promotion)	34
4.6.4	Retry & Monitoring	34
4.7	Traitements PySpark (notebooks Azure Synapse)	34
4.7.1	Notebook : Bronze_To_Silver.ipynb	34
4.7.2	Notebook : DataQuality_Checks.ipynb	34
4.8	Scripts SQL et couche Gold	35
4.9	Power BI — restitution	35
4.10	Logging, observabilité et artefacts	36
4.11	Exécution end-to-end : runbook synthétique	36
4.12	Paramètres et Configuration	37
4.13	Tests d'intégration et validation	37
4.14	Dépannage courant et solutions	38
4.15	Annexes techniques	38
4.16	Conclusion	38
Partie 5 — Tests, Validation, Monitoring et Optimisation		40
5.1	Introduction	40
5.2	Objectifs des tests	40
5.3	Stratégie de tests	40
5.3.1	Tests unitaires / de développement	40
5.3.2	Tests d'intégration	40

5.3.3	Tests end-to-end (E2E)	41
5.3.4	Tests de non-régression	41
5.4	Plan de tests et scénarios	41
5.5	Data Quality (DQ) — règles & automatisation	41
5.6	Métriques à collecter	42
5.7	Résultats de tests manuels (exemples)	43
5.8	Monitoring et alerting en production	43
5.9	Plan de validation & tests d'acceptation (UAT)	43
5.10	Automatisation des tests	44
5.11	Recommandations post-tests	44
5.12	Conclusion	44
Partie 6	Déploiement, Exploitation, Gouvernance et Conclusion	46
6.1	Introduction	46
6.2	Rappels du Cahier des Charges	46
6.3	Plan de déploiement (sans CI/CD)	46
6.3.1	Étapes de provisionnement et configuration	46
6.3.2	Configuration des accès et sécurité	47
6.4	Exploitation et runbook opérationnel	48
6.4.1	Opérations quotidiennes	48
6.4.2	Procédure en cas d'incident	48
6.4.3	Tâches hebdomadaires / mensuelles	48
6.5	Monitoring et alerting	48
6.6	Backlog fonctionnel et critères d'acceptation	49
6.6.1	Epic 1 : Infrastructure et Sécurité	49
6.6.2	Epic 2 : Ingestion (ADF)	49
6.6.3	Epic 3 : Transformation & Modélisation (Synapse)	49
6.6.4	Epic 4 : Power BI	49
6.6.5	Epic 5 : Monitoring & Quality	50
6.7	Gestion des bugs potentiels et actions préventives	50
6.8	Planning révisé et réalisation (alignement Cahier des Charges)	50
6.9	Limites du projet et recommandations finales	51
	Webographie	52
	Conclusion Générale	55

Remerciements

Je remercie Smartovate Ltd pour l'initiative pédagogique et pour le sujet fourni dans le cadre de la formation, ainsi que M. Abdelkhalek Bakkari, Fondateur et CEO de Smartovate, pour son encadrement professionnel et la mise à disposition des ressources techniques nécessaires à la réalisation de ce projet. Je remercie également Mme Loubna Ourabah (EIDIA, UEMF) pour son encadrement académique. Mes remerciements s'adressent enfin à tous les collègues et intervenants qui ont contribué par leurs retours aux validations et aux tests durant le projet.

Remarque : les jeux de données utilisés dans ce projet (Oracle SCM) sont des jeux simulés générés par l'auteur à des fins d'expérimentation et de démonstration (voir `DataGeneration.py` dans le dépôt).

Résumé

Ce rapport décrit la conception, l'implémentation et la validation d'un pipeline de données Cloud déployé sur Microsoft Azure. Le pipeline automatise l'ingestion de fichiers sources (snapshots Oracle SCM simulés), applique des transformations PySpark dans Azure Synapse pour produire des niveaux Bronze / Silver / Gold stockés sur ADLS Gen2, et expose les jeux de données optimisés pour Power BI. Le travail inclut : scripts de génération de données, orchestration ADF, notebooks PySpark, scripts de contrôle d'ingestion (ingestion_status), sécurité (Managed Identity, Key Vault) et tests de qualité des données.

Mots-clés : Azure Data Lake, Azure Data Factory, Azure Synapse, PySpark, Power BI, Medallion architecture, Oracle SCM (simulé), data quality.

Introduction Générale

Contexte

La montée en charge des besoins analytiques impose une plateforme de données capable d'ingérer, transformer et servir des données de manière répétable et traçable. Smartovate a fourni le brief de projet et les ressources pédagogiques ; l'étudiant a, de son côté, simulé les données Oracle SCM et développé la chaîne complète d'ingestion et de traitement.

Problématique

Ce PFA vise à résoudre les problématiques suivantes :

- Processus d'ingestion automatisé et résilient pour snapshots métier (fichiers plats).
- Nettoyage, transformation et historisation selon le pattern Medallion (Bronze / Silver / Gold).
- Mise à disposition de jeux de données performants et documentés pour Power BI.
- Respect des bonnes pratiques Cloud (sécurité, secrets, identité managée, coût).
- Validation de la qualité des données et mise en place d'indicateurs DQ.

Objectifs

Les objectifs du rapport sont :

1. Documenter la conception et l'architecture d'une pipeline Azure adapté au pattern Medallion.
2. Implémenter les composants clés (scripts de génération, ADF pipelines, notebooks PySpark, SQL Gold).
3. Mettre en place un mécanisme d'audit et de contrôle (ingestion_status, logs, quality gates).
4. Mesurer et valider la qualité et la performance de la solution.
5. Fournir un guide d'exploitation et des annexes (extraits de code, captures d'écran).

Partie 1

Présentation de l'organisation

Smartovate — rôle dans le projet

1.1 Smartovate : présentation de l'entreprise

Smartovate Ltd est une entreprise technologique et partenaire Microsoft/AWS, fondée à Londres en 2024 par **Abdelkhalek Bakkari**. Elle propose des solutions d'automatisation par intelligence artificielle, du déploiement Cloud, des formations spécialisées, ainsi que sa plateforme d'apprentissage adaptatif hybride **Subul**, principalement active dans la région du Golfe (GCC).

Domaines d'intervention

- **Intelligence Artificielle et Cloud** : intégration d'agents IA, gestion et déploiement de solutions sur Microsoft Azure et Amazon Web Services (AWS).
- **Plateforme Subul** : système d'apprentissage hybride et intelligent propulsé par l'IA, conçu pour personnaliser les parcours éducatifs et fournir des analyses en temps réel.
- **Formations professionnelles** : cours axés sur l'IA, le Cloud et la cybersécurité, disponibles en français et en anglais.
- **Secteurs ciblés** : transport, santé, tourisme et éducation.

Informations clés

TABLE 1.1 – Fiche d'identité — Smartovate Ltd

Champ	Valeur
Fondateur & CEO	Abdelkhalek Bakkari
Année de création	2024
Siège social	Londres, Royaume-Uni
Statut	Partenaire Microsoft / AWS
Produit phare	Subul — plateforme d'apprentissage adaptatif hybride
Zone d'activité principale	Région du Golfe (GCC)
Site officiel	https://www.smartovate.com

Dans le cadre de ce PFA, Smartovate est intervenue comme organisme de formation et commanditaire pédagogique : elle a fourni le sujet, l'environnement de formation et les ressources documentaires nécessaires. Le développement, la génération des données et l'implémentation technique ont été réalisés par l'étudiant.

1.2 Mission pédagogique

- Proposer un sujet de projet réaliste (intégration / transformation / restitution).
- Fournir un cadre et des ressources pour l'apprentissage des services Azure.
- Évaluer la solution livrée et la qualité du travail de l'étudiant.

1.3 Rôle de l'étudiant

L'étudiant a :

- Généré des jeux de données Oracle SCM simulés à l'aide de `DataGeneration.py`.
- Développé les scripts d'ingestion et d'upload (`AutomaticUpload.py`).
- Conçu et implémenté les pipelines ADF et les notebooks PySpark dans Synapse.
- Réalisé les transformations Bronze → Silver → Gold et préparé des datasets Power BI.
- Documenté et testé la solution (quality gates, latence, stabilité).

Partie 2

Analyse des Besoins Fonctionnels

Analyse des besoins

2.1 Introduction

Cette partie formalise les besoins fonctionnels et non-fonctionnels pour la plateforme de traitement de données réalisée dans le cadre du PFA. Elle décrit les sources de données (Oracle SCM simulé), le mapping attendu entre les couches Bronze / Silver / Gold, les règles de qualité de données et les engagements (SLA / performances).

2.2 Périmètre et Contexte

Le périmètre fonctionnel couvre :

- L'ingestion automatisée de snapshots issus d'un système Oracle SCM simulé (fichiers CSV générés par `DataGeneration.py`).
- Le traitement par lot pour produire des couches Bronze, Silver et Gold sur ADLS Gen2.
- L'exposition de tables/vues optimisées pour la restitution Power BI.
- La supervision et le contrôle d'ingestion (fichier `ingestion_status.csv` et logs).

Remarque : Smartovate est le commanditaire pédagogique ; toutes les données utilisées sont simulées et générées par l'auteur pour les besoins de la démonstration.

2.3 Sources de données (Oracle SCM simulé)

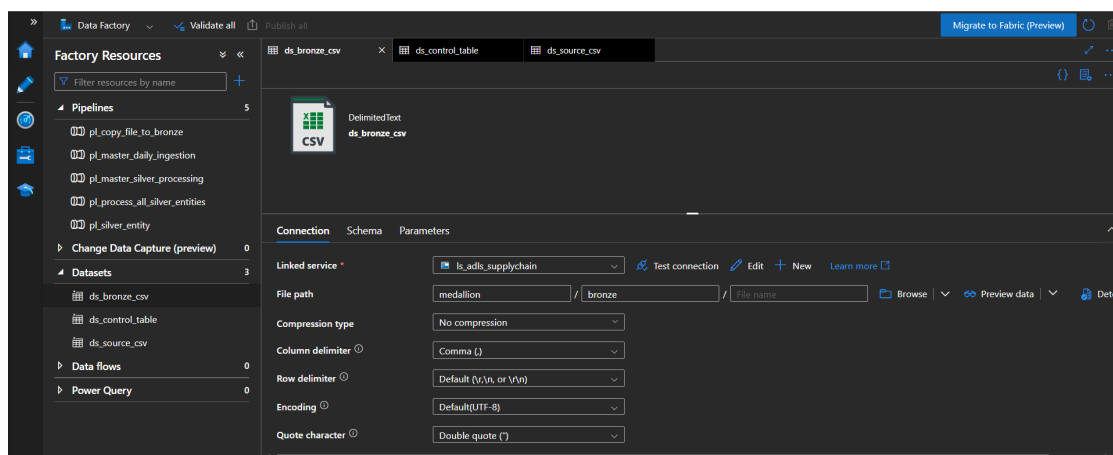
Les données sources correspondent à des exports/snapshots de l'ERP SCM simulé. Les scripts du dépôt génèrent les entités suivantes :

TABLE 2.1 – Fichiers sources générés (Oracle SCM simulé)

Fichier	Contenu
<code>suppliers.csv</code>	Fournisseurs (<code>supplier_id</code> , <code>name</code> , <code>country</code> , <code>category</code> , ...)
<code>purchase_orders.csv</code>	Commandes (<code>po_id</code> , <code>supplier_id</code> , <code>order_date</code> , <code>amount</code> , <code>status</code> , ...)
<code>deliveries.csv</code>	Livraisons (<code>delivery_id</code> , <code>po_id</code> , <code>delivered_date</code> , <code>qty</code> , <code>status</code> , ...)
<code>inventory.csv</code>	Stocks (<code>product_id</code> , <code>location</code> , <code>qty_onhand</code> , <code>snapshot_date</code> , ...)

Fichiers et utilitaires pertinents dans le dépôt :

- `DataGeneration.py` — script de génération des jeux de données simulés Oracle SCM.
- `AutomaticUpload.py` — utilitaire d'upload vers un conteneur source sur ADLS (ou environnement de test).
- `ingestion_status.py` et `ingestion_status_upload.py` — génèrent et envoient le fichier de contrôle d'ingestion.
- Dossier Notebooks/ — notebooks PySpark pour Bronze → Silver et contrôles qualité.



Aperçu des fichiers sources déposés (image du dépôt)

2.4 Exigences fonctionnelles détaillées

Les exigences fonctionnelles prioritaires :

1. Ingestion automatisée : ADF doit détecter les nouveaux fichiers sources et lancer les pipelines d'ingestion.
2. Stockage Bronze : conserver l'original (raw) et les métadonnées d'ingestion (fichier, timestamp, checksum).
3. Transformation Silver : appliquer nettoyage, typage, normalisation, gestion des hordatages et des clés naturelles.
4. Gold / Modélisation : construire des dimensions (supplier, product, time) et des faits (fact_deliveries et fact_orders) prêtes pour BI.
5. Contrôle & audit : tenir un fichier `ingestion_status.csv` et des logs (ADF + Synapse) pour chaque exécution.
6. Exposition : rendre disponibles des vues SQL ou tables matérialisées consultables depuis Power BI (via Synapse SQL Pool ou Serverless).

2.5 Exigences non-fonctionnelles

- **Performance** : Latence batch acceptable — ingestion complète et transformation pour une journée < 60 minutes (scénario de démonstration).
- **Disponibilité** : Pipelines résilients ; mécanismes de retry sur échec (ADF retry policy).
- **Sécurité** : Utiliser Managed Identity + Key Vault pour l'accès à ADLS ; ne pas stocker de secrets en clair.
- **Scalabilité** : Traitements PySpark dimensionnés (pools Spark) pour supporter augmentation de volumes.
- **Coût** : Prévoir taille des pools Spark et fréquence d'exécution pour optimiser coûts (monitoring de consommation).
- **Qualité** : Définir seuils DQ (ex. taux d'acceptation > 98% des enregistrements valides) et quality gates automatisés.

2.6 Règles de qualité des données (Data Quality)

Règles implémentées dans les notebooks et scripts de contrôle :

- **Schéma attendu** : chaque fichier doit respecter le schéma contractuel (colonnes et types). Les colonnes manquantes déclenchent alerte.
- **Clés naturelles non-nulles** : `supplier_id`, `po_id`, `product_id` ne doivent pas être null.
- **Unicité** : Les identifiants primaires doivent être uniques dans la fenêtre d'ingestion (détecter doublons).
- **Plages valides** : dates dans un intervalle raisonnable, quantités ≥ 0 , montants non négatifs.
- **Seuils d'acceptation** : si >5% d'enregistrements rejetés, passer le quality gate en fail et empêcher promotion vers Gold.

Les résultats DQ sont écrits dans un artefact de contrôle (ex : `quality_gate_status.csv` exposé via Synapse) et visualisés dans un dashboard DataQuality (`images/DataQualityDash.png`).

2.7 Mapping Bronze / Silver / Gold

Exemple de mapping simplifié pour l'entité `purchase_orders` :

TABLE 2.2 – Mapping Purchase Orders — Bronze / Silver / Gold

Couche	Contenu	Traitements / Notes
Bronze (raw)	Fichier CSV brut stocké sous <code>bronze/purchase_orders.csv</code>	Copie exacte par ADF. Stockage en CSV direct sans modification de schéma.
Silver (cleaned)	Colonnes typées (<code>po_id</code> : string, <code>supplier_id</code> : string, <code>order_date</code> : date, <code>amount</code> : decimal, <code>status</code> : string)	Transformation Spark. Nettoyage, renommage, conversion des types, gestion des valeurs manquantes, écriture au format Parquet.
Gold (business)	FACT_ORDERS (<code>po_id</code> , <code>supplier_id</code> , <code>order_date</code> , <code>amount</code> , <code>status</code> , ...)	Modélisation en schéma en étoile (Serverless SQL). Agrégations pour Power BI (DirectQuery), clés de dimensions et rafraîchissement incrémental/complet.

Répéter la méthode pour `deliveries`, `suppliers`, `inventory` et `cost_lines` ; le dépôt contient un schéma cible Gold (`images/GoldShema.drawio.png`) qui illustre les dimensions et faits.

2.8 Flux d'orchestration et contrôles

Orchestration (ADF) :

- Pipeline master d'ingestion (`MasterIngestioDaily`) : détecte fichiers, lance copie vers Bronze (Copy Activity), met à jour `ingestion_status.csv`.
- Pipelines enfants : exécution des notebooks Synapse pour Bronze → Silver ; exécution des jobs SQL pour construire Gold.
- Pipeline master Silver → Gold : orchestre la promotion des datasets validés vers Gold (`MasterSilverToGold`).

Voir images : `images/MasterIngestioDaily.png`, `images/MasterSilverToGold.png`.

2.9 SLA et critères d'acceptation

Critères d'acceptation pour une exécution journalière :

- Ingestion complète des fichiers présents en source avant la fenêtre SLA (ex. 02 :00 UTC).
- Quality gate = PASS (taux d'erreur inférieur aux seuils) pour promotion Silver → Gold.
- Latence totale (ingestion + transformation + promotion) < 60 minutes pour un jeu de données de taille pédagogique (ex : 1–2 Go).
- Logs et métriques disponibles dans un emplacement partagé et dashboards (temps d'exécution, erreurs).

2.10 Histoires utilisateurs (User Stories)

- En tant que Data Analyst, je veux accéder à une table Gold `orders_fact` propre et partitionnée pour construire des rapports Power BI sans transformer les données.
- En tant que Data Engineer, je veux que le pipeline échoue proprement et notifie l'équipe en cas de règles DQ non respectées.
- En tant qu'exploitant, je veux pouvoir relancer un run ADF pour une date spécifique après correction des erreurs.
- En tant que responsable, je veux un dashboard DQ montrant l'évolution quotidienne des anomalies et des volumes ingérés.

2.11 Traçabilité et journaux

Pour chaque lot traité, conserver :

- Métadonnées d'ingestion : `filename`, `source_path`, `size`, `ingestion_ts`, `checksum`.
- Résultats DQ : nombre de lignes totales, acceptées, rejetées, raisons.
- Logs ADF & Synapse : status, durées, exceptions.
- Versioning des notebooks / scripts (référence commit SHA du dépôt).

2.12 Conclusion de l'analyse

L'analyse a défini un périmètre clair (sources simulées Oracle SCM), les exigences fonctionnelles et non-fonctionnelles, des règles DQ et un mapping Bronze → Silver → Gold reproductible. Le plan d'implémentation (Partie 3) s'appuiera sur ces règles et sur les artefacts présents dans le dépôt (scripts de génération et d'ingestion, notebooks et images d'architecture).

Partie 3

Conception de l'Architecture et Choix Techniques

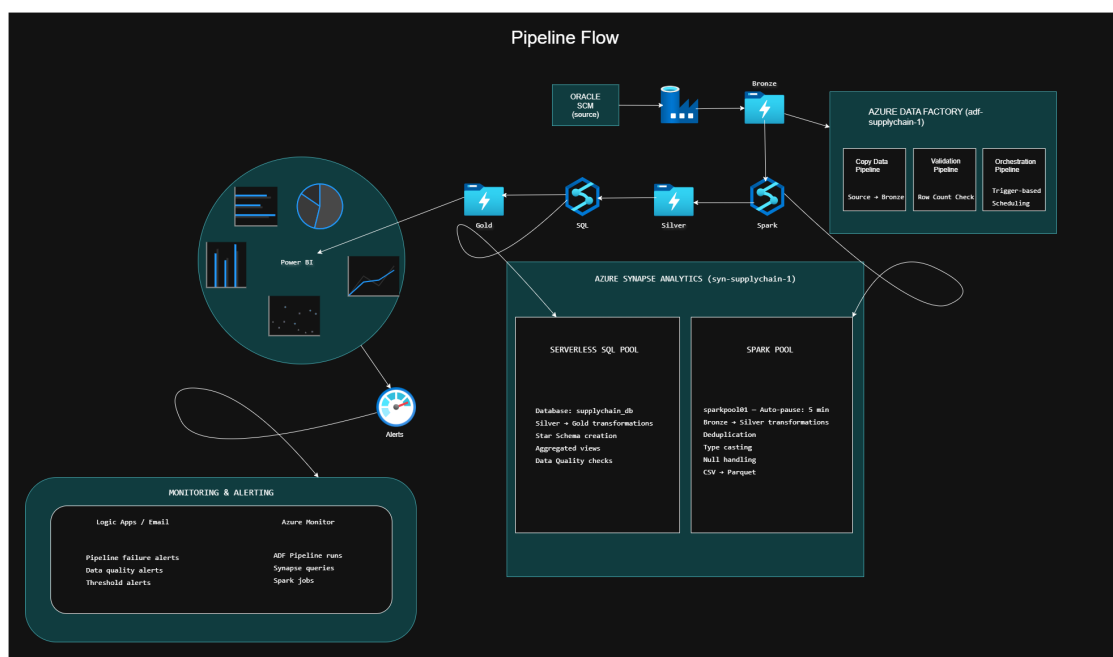
Conception détaillée

3.1 Introduction

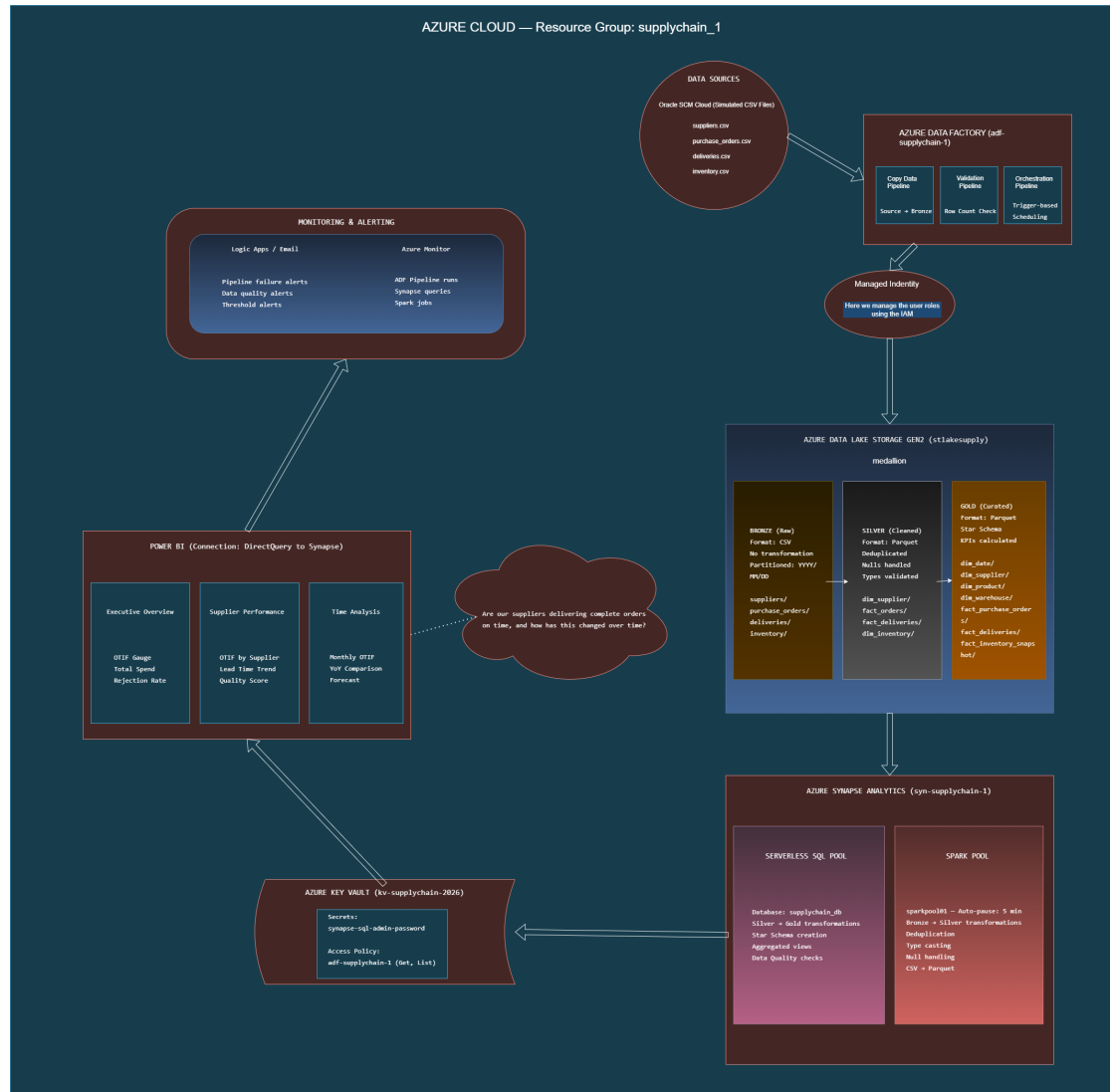
Cette partie décrit la conception détaillée de la plateforme de données implémentée pendant le PFA : architecture logique et physique, choix technologiques, organisation du Data Lake, conception des pipelines ADF, schéma des transformations (Bronze / Silver / Gold), sécurité et opérations. Les éléments cités renvoient aux artefacts présents dans le dépôt (scripts, notebooks, images).

3.2 Vue d'architecture logique

L'architecture logique suit le pattern Medallion (Bronze / Silver / Gold) et sépare clairement les étapes d'ingestion, de traitement et d'exposition :



Vue logique : orchestration et flux (image du dépôt)



Vue d'ensemble de l'architecture Azure (image du dépôt)

Composants principaux :

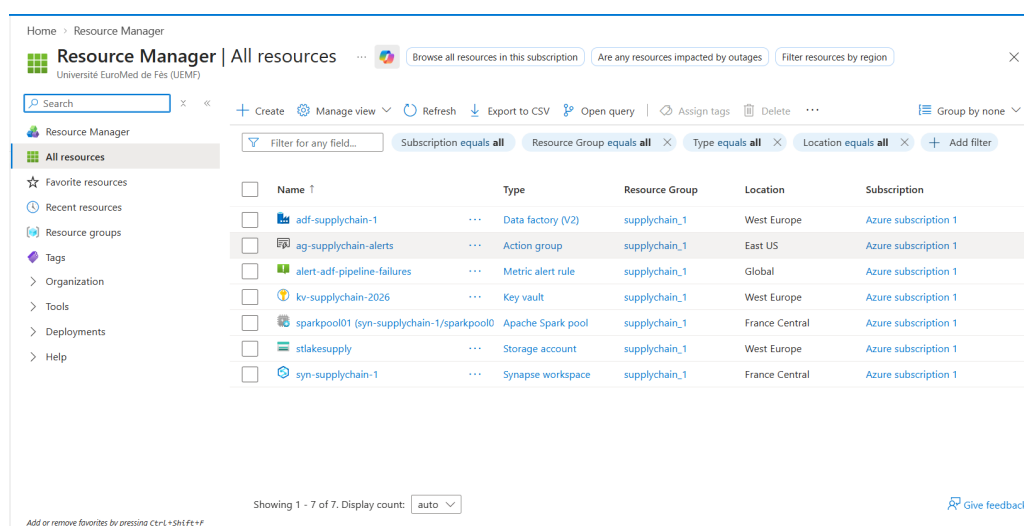
- Sources : snapshots CSV simulés (Oracle SCM) — générés localement via `DataGeneration.py`.
- Ingestion : Azure Data Factory (pipelines Master et enfants, Copy Activities).
- Stockage : ADLS Gen2 (zones raw/bronze, silver, gold).
- Traitement : Azure Synapse Spark pools (notebooks PySpark) pour transformations Bronze → Silver.
- Exposition : Synapse SQL (Serverless / Dedicated) pour vues et tables Gold, consommées par Power BI.
- Sécurité & secrets : Azure Key Vault, Managed Identities.
- Supervision : ADF runs, Synapse logs, Dashboards Power BI (`DataQualityDash.png`, `Alert.png`).

3.3 Architecture physique et ressources Azure

Ressources déployées (proposition et recommandation pédagogique) :

TABLE 3.1 – Ressources Azure principales

Ressource	Rôle / Utilisation
Storage Account (ADLS Gen2)	Stockage permanent : /raw, /bronze, /silver, /gold; ACLs basées sur Managed Identities
Azure Data Factory (ADF)	Orchestration : pipelines master/enfants, triggers, monitoring, retries
Azure Synapse Workspace	Spark pools pour transformation, Serverless SQL pour exposition, notebooks et linked services
Azure Key Vault	Stockage sécurisé des secrets et identifiants (si nécessaires)
Managed Identity (ADF / Synapse)	Authentification sans mot de passe pour accès ADLS et Key Vault
Power BI Service	Consommation des datasets exposés par Synapse SQL
Log Analytics / Monitor	Centralisation des métriques et alertes (ADF + Synapse)



Groupe de ressources Azure déployées (image du dépôt)

Nomination et environnement :

- **Resource groups** : supplychain_1
- **Storage account** : stlakesupply
- **Synapse workspace** : syn-supplychain-1
- **Apache Spark pool** : sparkpool01
- **ADF instance** : adf-supplychain-1
- **Key Vault** : kv-supplychain-2026
- **Governance** : purview-supplychain-prod
- **Monitoring & Alerts** : alert-adf-pipeline-failures, ag-supplychain-alerts

3.4 Organisation du Data Lake (Structure Medallion)

Schéma de partitionnement et layout de la solution :

- **/source-files/** — Zone d'atterrissage des fichiers bruts CSV (`deliveries.csv`, `inventory.csv`, `purchase_orders.csv`, `suppliers.csv`).
- **/medallion/bronze/** — Copies d'ingestion brutes conservées au sein du conteneur Medallion.
- **/medallion/silver/{domain}/year=YYYY/month=MM/** — Données nettoyées et structurées par domaine métier (`deliveries`, `inventory`, `purchase_orders`, `suppliers`) avec partitionnement Hive (ex. `year=2026/month=07/`).
- **/medallion/gold/{table}/year=YYYY/month=MM/** — Schéma en étoile modélisé pour la BI :
 - **Tables de faits** : `fact_deliveries`, `fact_orders` (partitionnées par date métier)
 - **Tables de dimensions** : `dim_date`

Home > Resource Manager | All resources > stlakesupply | Containers

medallion ...

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

medallion > bronze

Authentication method: Microsoft Entra user account (Switch to Access key)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 4 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[-]					...
☐	deliveries.csv	04/08/2026, 15:56:49	Cool (Inferred)	Block blob	13.43 KiB	Available
☐	inventory.csv	04/08/2026, 15:57:46	Cool (Inferred)	Block blob	3.92 KiB	Available
☐	purchase_orders.csv	04/08/2026, 15:55:50	Cool (Inferred)	Block blob	18.97 KiB	Available
☐	suppliers.csv	04/08/2026, 15:55:04	Cool (Inferred)	Block blob	9.81 KiB	Available

Add or remove favorites by pressing Ctrl+Shift+F

Structure de la zone Bronze sur ADLS Gen2 (image du dépôt)

Home > Resource Manager | All resources > stlakesupply | Containers

medallion ...

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

medallion > silver

Authentication method: Microsoft Entra user account (Switch to Access key)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 4 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[-]					...
☐	deliveries	23/07/2026, 22:23:54				...
☐	inventory	23/07/2026, 22:21:08				...
☐	purchase_orders	23/07/2026, 22:22:32				...
☐	suppliers	23/07/2026, 22:25:17				...

Add or remove favorites by pressing Ctrl+Shift+F

Organisation de la zone Silver — vue 1 (image du dépôt)

Home > Resource Manager | All resources > stlakesupply | Containers

medallion ...

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

medallion > silver > deliveries > year=2026

Authentication method: Microsoft Entra user account (Switch to Access key)

Search blobs by prefix (case-sensitive) Only show active objects

Showing all 2 items

	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[-]					...
☐	month=07	23/07/2026, 22:23:56				...
☐	month=08	04/08/2026, 16:04:29				...

Add or remove favorites by pressing Ctrl+Shift+F

Organisation de la zone Silver — vue 2 (image du dépôt)

Home > Resource Manager | All resources > stlakesupply | Containers

medallion ...

Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

medallion > gold

Authentication method: Microsoft Entra user account (Switch to Access key)

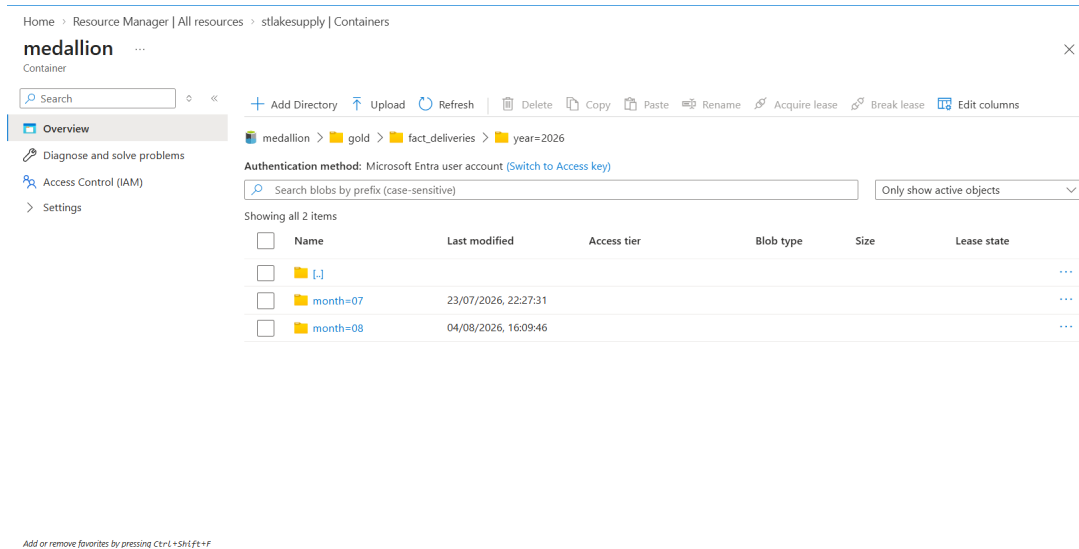
Search blobs by prefix (case-sensitive) Only show active objects

Showing all 3 items

	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[-]					...
☐	dim_date	27/07/2026, 13:01:09				...
☐	fact_deliveries	23/07/2026, 22:27:31				...
☐	fact_orders	23/07/2026, 22:27:29				...

Add or remove favorites by pressing Ctrl+Shift+F

Organisation de la zone Gold — vue 1 (image du dépôt)



Organisation de la zone Gold — vue 2 (image du dépôt)

3.5 Design des pipelines ADF

Topologie des pipelines ADF (implémentation pédagogique) :

— **Pipeline Master : MasterIngestioDaily**

- Trigger : schedule (daily) ou manuel pour les tests.
- Get Metadata / ForEach sur les fichiers entrants.
- Copy Activity : copie source → bronze (raw → bronze store).
- Mise à jour de `ingestion_status.csv`.
- En cas de succès : appel du pipeline enfant pour la transformation Bronze → Silver (exécution notebook dans Synapse).
- En cas d'échec : journalisation de l'erreur, notification (optionnel via LogicApp / Teams / Email).

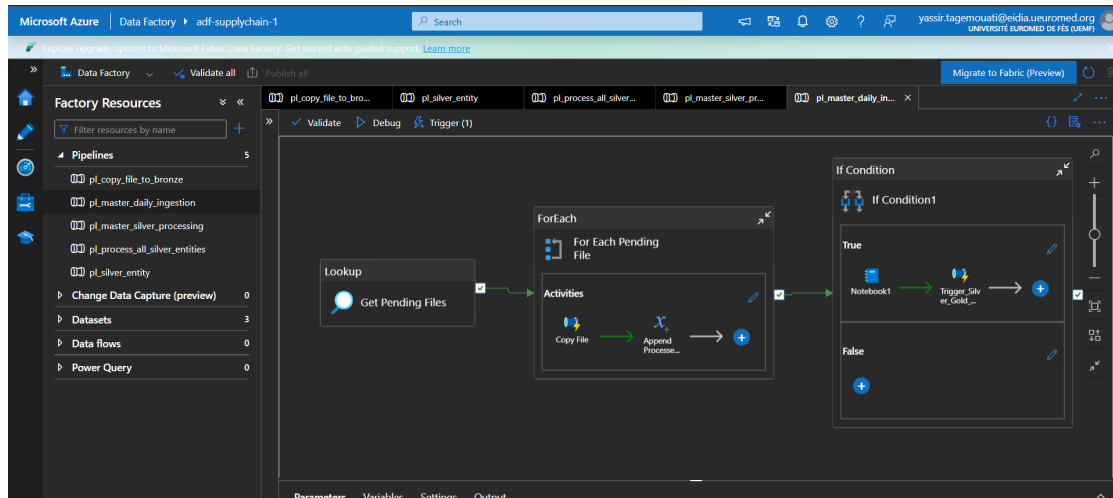
— **Pipeline BronzeToSilver (enfant)**

- Exécute un notebook Synapse (PySpark) par entité (paramétré).
- Après le notebook : exécution des contrôles DQ (notebook ou SQL).
- Mise à jour de l'artefact quality gate (`quality_gate_status.csv`).

— **Pipeline SilverToGold (promotion)**

- Exécute les scripts SQL / procédures stockées pour construire les tables de dimensions et de faits Gold.
- S'assure que le quality gate est PASS avant la promotion.

Illustration (dossier images) :



Master ingestion pipeline (image du dépôt)

Name ↑↓	Type ↑↓
ls_adls_supplychain	Azure Data Lake Storage Gen2
ls_keyvault_supplychain	Azure Key Vault
ls_synapse_notebook	Azure Synapse Analytics (Artifacts)
ls_synapse_serverless_sql	Azure Synapse Analytics
ls_synapse_supplychain	Azure Synapse Analytics

Linked services configurés dans Azure Data Factory (image du dépôt)

3.6 Traitements et transformations (Synapse PySpark)

Principaux traitements implémentés dans les notebooks :

1. Lecture Bronze : lire Parquet/CSV et récupérer les métadonnées d'ingestion (`filename`, `ingestion_ts`, `source`).
2. Normalisation : uniformiser les formats (dates, nombres), trimming, remplacement des valeurs invalides.
3. Validation & filtrage : appliquer les règles DQ (clés non nulles, plages valides).
4. Enrichissement : jointures opportunistes (ex : supplier lookup), calculs dérivés (taux, lead time).
5. Déduplication : conserver le dernier snapshot par clé naturelle (order by `ingestion_ts`), utiliser des window functions.
6. Écriture Silver : écrire en Parquet partitionné par date métier (ex : `order_date`) et compaction si nécessaire.

7. Promotion vers Gold : agrégation et création de clés substituts (surrogate keys), matérialisation de tables pour la BI.

Exemples techniques :

- Déduplication PySpark : `row_number() over(partition by natural_key order by ingestion_ts desc)` puis filtre `row_number = 1`.
- Gestion des clés substituts : générateur de surrogate key via `monotonically_increasing_id` et persistance dans la table de dimension.
- Stockage : Parquet avec compression Snappy pour l'équilibre coût/performance.

Le dépôt contient des notebooks illustrant ces étapes dans le dossier `Notebooks/` (notebooks `bronze_to_silver`, notebooks de qualité).

3.7 Schéma Gold — Modélisation pour BI

Le modèle Gold suit un schéma en étoile (star schema) : dimensions (supplier, product, date, status) et faits (`orders_fact`, `deliveries_fact`, `inventory_fact`). Voir schéma de référence fourni dans le dépôt :

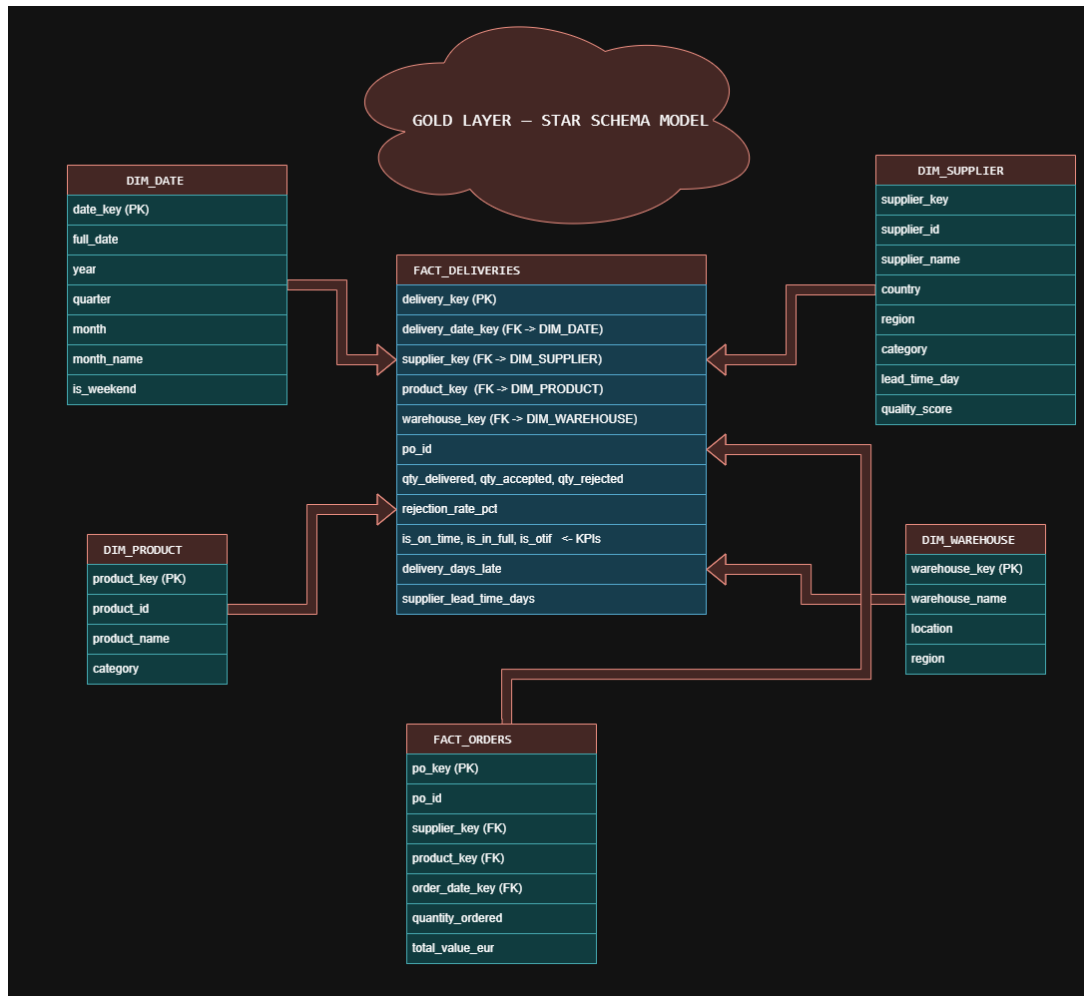


Schéma Gold (image du dépôt)

3.8 Conception des Dimensions et Modélisation Gold

- **Dimensions en zone Gold** : Seule la dimension temporelle commune `dim_date` est matérialisée dans la zone Gold pour garantir la cohérence des jointures temporelles et du time-intelligence.
- **Consommation directe depuis la zone Silver** : Pour éviter la recreation complète et le recalcul coûteux de toutes les dimensions applicatives, les attributs des entités métiers (`suppliers`, `inventory`, etc.) sont directement exploités depuis la zone Silver sans duplication inutile dans Gold.
- **Tables de faits** : `fact_deliveries` et `fact_orders` sont matérialisées dans la zone Gold, partitionnées par date métier (`year=YYYY/month=MM/`) pour optimiser les performances analytiques et l'actualisation incrémentale.

3.9 Sécurité et gestion des accès

Bonnes pratiques appliquées :

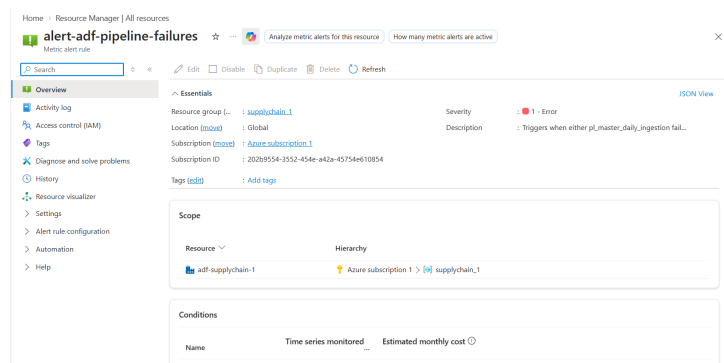
- Utiliser des Managed Identities (ADF, Synapse) pour l'accès à ADLS — éviter les clés en clair.
- Key Vault pour stocker les secrets éventuels (service principal, credentials externes).
- Contrôle d'accès : ACLs sur les conteneurs ADLS, RBAC sur les ressources Azure.
- Isolation des environnements : dev / test / prod (resource groups et nommage).

3.10 Monitoring, logging et alerting

Plan de supervision :

- ADF Monitoring : pipeline runs, activity runs.
- Synapse Monitoring : notebook runs, logs driver/executor Spark.
- Log Analytics : centraliser les logs et définir des règles d'alerte (échecs, exécutions longues).
- Data Quality Dashboard : exposer `quality_gate_status` via Synapse SQL et le visualiser dans Power BI (`images/DataQualityDash.png`).
- Alerting : notifications (email/Teams) en cas d'échecs critiques (ex : quality gate FAIL).

Illustration d'alerte (exemple pédagogique) :



Exemple de widget d'alerte (image du dépôt)

3.11 Opérations et runbook (exploitation)

Procédure opérationnelle (runbook) résumée :

1. Vérifier l'état des triggers ADF (scheduling) chaque matin.

2. En cas d'échec : consulter les logs ADF Activity, puis les logs des notebooks Synapse.
3. Si erreur de source (format ou schéma) : isoler le fichier dans `/raw/_quarantine` et corriger la source (ou signaler au commanditaire).
4. Relancer le run ADF pour la date concernée via paramétrage (partition de date).
5. Mettre à jour `ingestion_status.csv` pour refléter l'état (script `ingestion_status_upload.py`).
6. Vérifier `quality_gate_status` et, si PASS, autoriser la promotion vers Gold.

3.12 CI/CD et Infrastructure as Code (recommandations)

Pour industrialiser le déploiement :

- Versionner les notebooks et scripts dans le dépôt Git (déjà présent) ; référencer le commit SHA dans les métadonnées d'exécution.
- Déployer les artefacts ADF/Synapse via des templates ARM / pipelines Azure DevOps (publication depuis le dépôt).
- Utiliser Terraform / Bicep pour provisionner l'infrastructure (storage account, workspace Synapse, instance ADF).
- Automatiser les tests d'intégration (petits datasets) dans le pipeline CI avant promotion en production.

3.13 Performances, dimensionnement et coûts

Recommandations pour les pools Spark (pédagogique vs production) :

- Dev / Démo : petit pool (2–4 nœuds), faible coût, adapté aux datasets < 5 Go.
- Prod : configurer l'autoscale des pools et la concurrence des jobs, mesurer l'usage mémoire/exécuteurs à partir des runs.
- Stockage : Parquet + partitionnement (par date) pour minimiser les volumes scannés et réduire les coûts pour Serverless SQL.

Estimation indicative des coûts mensuels réalisée avec les calculateurs officiels Azure :

Azure Data Lake						
Service category	Service type	Custom name	Region	Description	Estimated monthly cost	Estimated upfront cost
Storage	Storage Accounts	stalesupply	West Europe	Data Lake Storage Gen2, Standard, LRS Redundancy, Cool Access Tier, Hierarchical Namespace File Structure, 10 GB Capacity - Pay as you go. Write operations: 4 MB x 0 operations, Read operations: 4 MB x 0 operations, 10 Iterative read operations, 100,000 Archive High Priority Read, 5 Iterative write operations, 10 Other operations, 2 GB Data Retrieval, 1,000 GB Archive High Priority Retrieval, 0 GB Data Write	\$0.49	\$0.00
Analytics	Azure Data Factory	adf-supplychain-1	West Europe	Azure Data Factory V2 Type: Data Pipeline Service Type, Azure Integration Runtime: 0 Activity Runs, 4 Data movement units, 0 Pipeline activities, 0 Pipeline activities - External	\$1.18	\$0.00
Security	Key Vault	kv-supplychain-2028	West Europe	Vault: 1 operations, 0 advanced operations, 0 renewals, 0 protected keys, 0 advanced protected keys, Managed HSM Pools: 0 Standard \$1 HSM Pools, x 730 Hours	\$0.03	\$0.00
Databases	Azure Synapse Analytics	syn-supplychain-1	France Central	Tier: Compute Optimized Gen2, Dedicated SQL Pools: DWU 100 x 0 Hours, 0 TB of storage with Geo-redundant disaster recovery, Serverless SQL Pools: 0.0012 TB of data queried, Apache Spark Pools: Memory optimized: (3 Instances), Small-sparkpool x 10 Hours, France Central Region, 0 GB of data collected per day, 1 days of Hot Cache, 1 days of total retention, 1 times estimated data compression, 0 Hours of 2 x Extra Small (2 vCores) Engine Instances, 0 Hours of 2 x 1 vCore Data Management Instances	\$18.39	\$0.00
Support			Support		\$0.00	\$0.00
			Licensing Program	Microsoft Customer Agreement (MCA)		
			Billing Account			
			Billing Profile			
			Total		\$20.09	\$0.00

Détail de l'estimation de coût Azure retenue pour le projet (image du dépôt)

3.14 Annexes et références dans le dépôt

Les éléments techniques et artefacts de conception se trouvent dans le dépôt :

- `DataGeneration.py`, `AutomaticUpload.py` — génération & ingestion des jeux de données (Oracle SCM simulé).
- `ingestion_status.py` / `ingestion_status_upload.py` — contrôle d'ingestion.
- `Notebooks/` — implémentations Spark.
- `SQL scripts/` — scripts Gold et vues.
- `images/` — diagrammes d'architecture et captures d'écran.

3.15 Conclusion

La conception proposée combine architecture logique (Medallion), orchestration ADF, traitements PySpark et exposition Synapse/Power BI. Elle met l'accent sur la traçabilité (`ingestion_status`), la qualité des données (quality gates) et la sécurité (Managed Identity & Key Vault). La partie suivante (Partie 4) décrira l'implémentation concrète, les notebooks, les scripts et les extraits de code mis en œuvre dans le dépôt.

Partie 4

Implémentation de la Plateforme

Implémentation et déploiement

4.1 Introduction

Cette partie décrit l'implémentation concrète du pipeline : préparation des environnements, scripts de génération et d'ingestion, configuration des pipelines Azure Data Factory (ADF), notebooks Synapse PySpark pour Bronze → Silver, scripts SQL pour Gold, et construction des rapports Power BI. Les références aux fichiers et images renvoient au dépôt Azure_Cloud.

4.2 Environnements et prérequis

Avant d'exécuter ou de tester la solution, préparer :

- Un abonnement Azure (ou un environnement de formation fourni).
- Un Storage Account ADLS Gen2 (conteneurs : source/raw, bronze, silver, gold, control).
- Un workspace Azure Synapse (Spark pool configuré).
- Une instance Azure Data Factory (ADF).
- Azure Key Vault (optionnel mais recommandé) et Managed Identities pour ADF/-Synapse.
- Power BI Service (compte) pour publier les rapports.

Fichiers & dossiers clés du dépôt :

- `DataGeneration.py` — génération de jeux simulés Oracle SCM.
- `AutomaticUpload.py` — upload automatisé des fichiers source vers ADLS / conteneur source.
- `ingestion_status.py`, `ingestion_status_upload.py` — génération et upload du fichier de contrôle d'ingestion.
- `Notebooks/` — notebooks PySpark (Bronze → Silver, contrôles DQ).
- `SQL scripts/` — scripts pour la couche Gold (vues, CETAS, tables).
- `images/` — diagrammes : `MasterIngestionDaily.png`, `CopySinkToBronze.png`, `MasterSilverToGoldShema.drawio.png`, `DataQualityDash.png`, etc.
- `README.md` et `steps1.docx` .. `steps4.docx` — documentation pas-à-pas.

4.3 Génération des données (Oracle SCM simulé)

Le dataset utilisé pour les démonstrations est simulé par l'auteur :

- Ouvrir et exécuter `DataGeneration.py` (Python 3.8+). Le script génère plusieurs CSV (`suppliers`, `purchase_orders`, `deliveries`, `inventory`) dans un dossier local (par ex. `supplyChain_db/`).

- Paramètres possibles : nombre de jours à simuler, volume de lignes par entité, seed RNG, chemins de sortie. Voir l'en-tête du script pour les variables modifiables.

But de l'étape :

- Produire des snapshots journaliers représentatifs d'un système Oracle SCM.
- Rendre possible la génération incrémentale afin d'émuler l'arrivée de nouveaux fichiers source.

Exemple d'utilisation locale :

```
python DataGeneration.py -days 7 -out ./supplyChain_db
```

4.4 Téléversement initial vers ADLS (AutomaticUpload.py)

`AutomaticUpload.py` est un utilitaire simple qui prend les fichiers générés localement et les copie vers le conteneur `source/` du Storage Account. Il peut être exécuté localement (pour tests) ou remplacé par une ingestion réelle via ADF.

- Paramètres importants : `account_name`, `container`, `credentials` (privilégier Managed Identity ou un jeton SAS stocké dans Key Vault).
- Comportement : conserve l'arborescence locale, ajoute un horodatage de chargement si nécessaire.

Notes de sécurité :

- Ne jamais committer de secrets dans le dépôt ; utiliser Key Vault et des variables d'environnement.

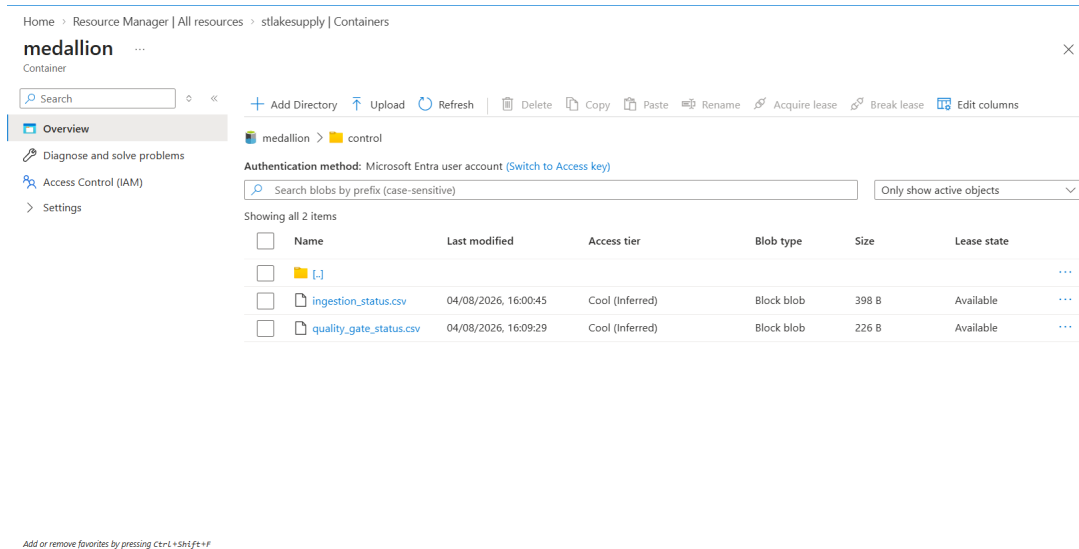
4.5 Contrôle d'ingestion (ingestion_status)

Le dépôt inclut deux scripts d'aide :

- `ingestion_status.py` — génère un fichier `ingestion_status.csv` contenant les colonnes attendues : `entity`, `filename`, `processing_date`, `status` (Pending/Processed).
- `ingestion_status_upload.py` — envoie `ingestion_status.csv` dans `/medallion/control/` sur ADLS afin que le pipeline ADF l'utilise comme source de contrôle.

Usage :

- Exécuter `ingestion_status.py` après génération des fichiers pour initialiser l'état.
- Le pipeline ADF Master lit ce fichier pour orchestrer les copies et la coordination.



Fichiers de contrôle d'ingestion dans le conteneur /medallion/control/ (image du dépôt)

4.6 Orchestration : Azure Data Factory (ADF)

L'orchestration centrale a été conçue en trois niveaux.

4.6.1 MasterIngestioDaily (pipeline maître)

Objectif : détecter les fichiers entrants et pousser les copies vers Bronze (raw), puis déclencher les transformations.

Étapes principales :

1. Get Metadata / Lookup `ingestion_status.csv` (control).
2. ForEach sur les éléments Pending → Copy Activity (source blob → chemin Bronze).
3. En cas de succès : appel du pipeline `BronzeToSilver` (Execute Pipeline activity) avec paramètres (`entity`, `date`, `filename`).
4. Mise à jour de `ingestion_status.csv`.

Visualisation :

- Voir image : `images/MasterIngestioDaily.png`
- Voir le détail de la Copy Activity : `images/CopySinkToBronze.png` et `images/FirstSink.png`

4.6.2 BronzeToSilver (pipeline enfant)

- Exécute un notebook Synapse pour transformer et nettoyer l'entité paramétrée.
- Paramètres : `entity_name`, `processing_date`, `file_path`, `storage_account`.
- Contrôles post-notebook : exécution d'un notebook DQ qui produit `quality_gate_status.csv`.

4.6.3 MasterSilverToGold (promotion)

- Après validation DQ (quality gate PASS), ce pipeline déclenche les scripts SQL dans Synapse pour construire / mettre à jour les tables Gold.
- Les activités incluent : Stored Procedure Activity / Script Activity (exécution de GoldShema.sql et des procédures).

4.6.4 Retry & Monitoring

- Activités ADF configurées avec une politique de retry (ex : retry = 3, retryIntervalInSeconds = 30).
- Notifications (optionnel) via Logic Apps / Email / Teams en cas d'échec critique.

4.7 Traitements PySpark (notebooks Azure Synapse)

Les transformations sont implémentées sous forme de notebooks PySpark dans Synapse. Principaux notebooks présents dans le dépôt (Notebooks/) :

4.7.1 Notebook : Bronze_To_Silver.ipynb

Lecture du fichier bronze (Parquet/CSV), puis application des opérations :

- Normalisation des colonnes (typage, trimming).
- Conversion des dates (string → date).
- Remplacement des valeurs aberrantes (ex : montants négatifs).
- Déduplication : window function `row_number()` over (partition by natural_key order by ingestion_ts desc) puis filtre `row_number = 1`.
- Écriture Silver en Parquet partitionné (par ex. `order_date`).

Extrait logique (pseudo-code PySpark) :

- `df = spark.read.parquet(bronze_path)`
- `df = df.withColumn('order_date', to_date(col('order_date'), 'yyyy-MM-dd'))`
- `window = Window.partitionBy('po_id').orderBy(desc('ingestion_ts'))`
- `df_dedup = df.withColumn('rn', row_number().over(window)).filter(col('rn')==1).drop('rn')`
- `df_dedup.write.mode('overwrite').partitionBy('order_date').parquet(silver_path)`

4.7.2 Notebook : DataQuality_Checks.ipynb

- Exécute les règles DQ listées en Partie 2.
- Calcule des métriques : `total_rows`, `valid_rows`, `rejected_rows`, `percent_rejected`.

- Écrit un artefact JSON/CSV `quality_gate_status` dans `/medallion/control/` pour consommation par ADF.

Exemple de quality gate :

- Si `percent_rejected` > 5% $\Rightarrow$ quality gate FAIL (pas de promotion).
- Sinon $\Rightarrow$ PASS.

4.8 Scripts SQL et couche Gold

Les scripts dans SQL `scripts/` définissent les étapes d'assemblage Gold :

- `GoldShema.sql` — DDL et création des vues / tables Gold (dimensions & faits).
- Scripts d'upsert pour les dimensions (logique SCD Type 2) et d'insertion pour les faits.
- Scripts de matérialisation / CETAS si besoin (`CREATE EXTERNAL TABLE AS SELECT`) pour optimiser l'accès Power BI.

Exécution :

- Les scripts sont exécutés depuis ADF via l'activité Synapse Notebook/Script, ou directement depuis Synapse Studio (SQL script).

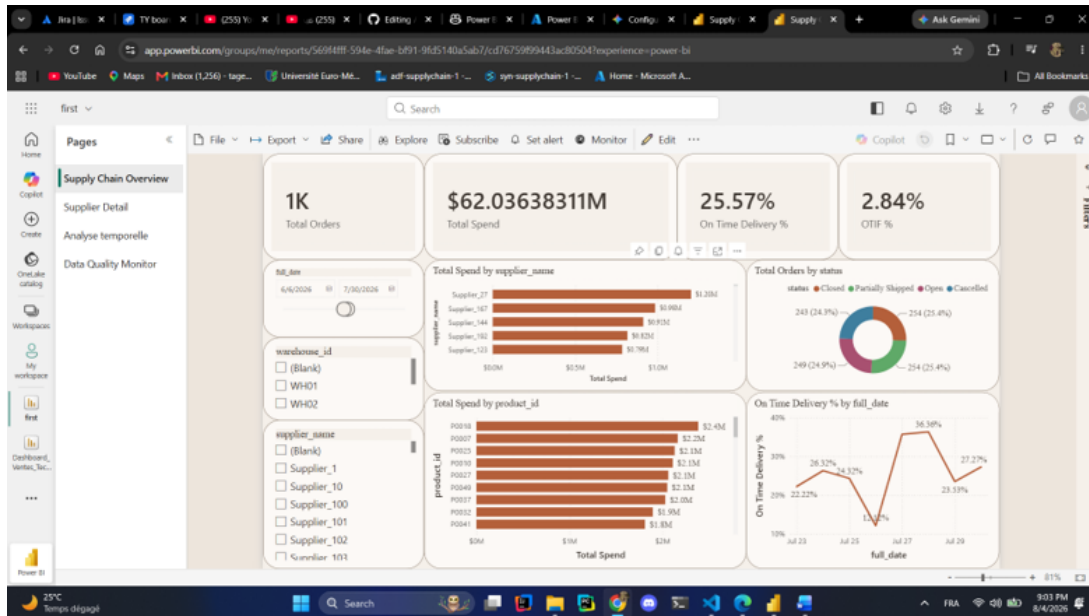
4.9 Power BI — restitution

Les artefacts produits :

- Datasets Power BI pointant sur Synapse Serverless / Dedicated SQL (Import pour les agrégats, DirectQuery pour les faits détaillés si acceptable).
- Rapports / Dashboards (images fournies : `images/SupplierOverviewDash.png`, `images/SupplierDetailsDash.png`, `images/AnalyseTempDash.png`, `images/CostDaily.png`).
- Dashboard Data Quality (`images/DataQualityDash.png`).

Actions clés :

- Publier les rapports dans un workspace Power BI Service lié au même tenant.
- Configurer la gateway.
- Utiliser Import pour les mesures KPI stables (meilleure performance), DirectQuery pour l'exploration en temps réel si besoin.



Workspace Power BI Service — rapports publiés (image du dépôt)

4.10 Logging, observabilité et artefacts

Chaque exécution crée / actualise :

- `ingestion_status.csv` — état courant des fichiers ingérés.
- `quality_gate_status.csv` — résultats DQ.
- Logs de run ADF — visibles via ADF Monitor.
- Logs des notebooks Synapse — logs driver/executor (stockés dans le storage lié).

4.11 Exécution end-to-end : runbook synthétique

Étapes pour lancer un run end-to-end (dev/test) :

1. Générer des données locales : `python DataGeneration.py -days N -out ./supplyChain_db`
2. Mettre à jour le contrôle d'ingestion : `python ingestion_status.py -source ./supplyChain_db -out ingestion_status.csv`
3. Uploader les fichiers : `python AutomaticUpload.py -local ./supplyChain_db -dest {adls_container}`
4. Uploader le statut d'ingestion : `python ingestion_status_upload.py -file ingestion_status.csv -dest /medallion/control/`
5. Lancer le trigger ADF (ou exécuter le pipeline maître manuellement).
6. Vérifier le run ADF Master → contrôler les écritures Bronze.
7. Vérifier les runs du notebook BronzeToSilver → contrôler les écritures Silver.

8. Vérifier le notebook DataQuality → `quality_gate_status` dans `/medallion/control/`.
9. Si PASS → ADF MasterSilverToGold exécute les scripts Gold.
10. Vérifier les tables Gold via Synapse Studio ; actualiser les rapports Power BI.

4.12 Paramètres et Configuration

Paramètres d'environnement à documenter (dans le README & le runbook) :

- **Compte de stockage & Conteneurs** : `stlakesupply` (conteneurs `source-files` et `medallion`).
- **Workspace Synapse & Spark Pool** : `syn-supplychain-1`, pool Spark `sparkpool101`.
- **Instance Azure Data Factory** : `adf-supplychain-1` (pipelines d'ingestion et de transformation Medallion).
- **Gestion des secrets & Sécurité** : `kv-supplychain-2026` (secrets pour chaînes de connexion, clés d'accès et jetons SAS pour ADLS Gen2).
- **Colonnes de partitionnement par défaut** : partitionnement Hive par date métier avec les colonnes `year` et `month` (ex. `year=2026/month=07/`).
- **Gouvernance & Monitoring** : compte Microsoft Purview `purview-supplychain-prod`, règle d'alerte `alert-adf-pipeline-failures` et groupe d'action `ag-supplychain-alerts`.

Data Map > Data sources > syn-supplychain-1 >

scan-synapse-supplychain run history purview-supplychain-prod

purview-supplychain-prod

Run scan now Edit scan Delete scan Cancel runs Refresh Edit columns

Showing 9 items

Resource	Source type	Run ID	Status	Run type	Scan type	Assets discover...	Assets ingested	Start time	Ru
syn-supplyc...	Azure Synapse...	b87cb6f8-2cd...		Full scan	Manual	12	12	08/05/2026, 12:1...	10
supplych...	Azure Synapse...	6a4ca0b8-bea...	Completed	Full scan	Manual	3	3	08/05/2026, 12:1...	13
gold_db	Azure Synapse...	2124936c-65c...	Completed	Full scan	Manual	9	9	08/05/2026, 12:1...	10
syn-supplyc...	Azure Synapse...	cb01a154-149...		Full scan	Manual	12	12	08/05/2026, 11:2...	12
gold_db	Azure Synapse...	4525c6b1-317...	Completed with ex	Full scan	Manual	9	9	08/05/2026, 11:2...	13
supplych...	Azure Synapse...	d8b662cc-35c...	Completed	Full scan	Manual	3	3	08/05/2026, 11:2...	11
syn-supplyc...	Azure Synapse...	855fc3c9-73df...		Full scan	Manual	9	9	08/04/2026, 11:3...	16
gold_db	Azure Synapse...	d2cb4551-be6...	Completed with ex	Full scan	Manual	9	9	08/04/2026, 11:3...	16
supplych...	Azure Synapse...	225411d0-948...	Failed	Full scan	Manual	0	0	08/04/2026, 11:3...	17

Scan de gouvernance Microsoft Purview sur le workspace Synapse (image du dépôt)

4.13 Tests d'intégration et validation

- Tests unitaires sur de petites portions de données (notebooks de test).
- Tests d'intégration : exécution end-to-end sur un dataset d'exemple (1-2 Go).

- Vérifications post-run :
 - Comptages comparés Bronze → Silver → Gold (écarts attendus si rejets).
 - Métriques du quality gate (`percent_rejected`, répartition des raisons).
 - Latence globale (temps mesurés à partir des runs ADF & Synapse).

4.14 Dépannage courant et solutions

- Erreur de lecture ADLS : vérifier la Managed Identity / les ACLs et les liaisons Key Vault.
- Notebook timeout / OOM : augmenter la taille du pool Spark ou partitionner davantage.
- Échec DQ : isoler le(s) fichier(s) erroné(s) dans `/raw/_quarantine` et corriger la source ou les règles.
- Problèmes de rafraîchissement Power BI : vérifier les credentials du dataset et la gateway (si nécessaire).

4.15 Annexes techniques

Les annexes du rapport contiendront :

- Extraits clés de `DataGeneration.py` et `AutomaticUpload.py`.
- Extrait de notebook PySpark (déduplication & contrôles DQ).
- Extrait de `GoldShema.sql` (création des dimensions & faits).
- Copies des images de pipeline et des dashboards (`images/`).
- Pas-à-pas d'installation et d'exécution (fichiers `steps1..4.docx` et `README.md`).

4.16 Conclusion

L'implémentation illustre une plateforme pédagogique complète : génération de données simulées Oracle SCM, ingestion robuste, transformations PySpark, contrôles DQ automatisés et exposition BI. La mise en œuvre est reproductible et paramétrable ; les bonnes pratiques (secrets, identités managées, partitionnement, monitoring) sont recommandées pour toute industrialisation future.

Partie 5

Tests, Validation, Monitoring et Optimisation

Stratégie de tests et validation

5.1 Introduction

Cette partie décrit la stratégie de tests et de validation mise en place pour la pipeline Azure développée durant le PFA. Elle couvre les types de tests (unitaires, d'intégration, end-to-end), les tests de qualité des données, les métriques observables, les scénarios de charge et la supervision (monitoring & alerting). Les illustrations proviennent des artefacts du dépôt (dashboards et images).

5.2 Objectifs des tests

Les objectifs principaux :

- Vérifier la robustesse des pipelines ADF (résilience, retry, comportement en erreur).
- Garantir la conformité au schéma et aux règles de qualité (Data Quality).
- Valider la logique de transformation (déduplication, typage, enrichissement).
- Mesurer les performances (latence batch, temps de traitement Spark, coût estimé).
- Mettre en place la surveillance et les alertes pour l'exploitation.

5.3 Stratégie de tests

La stratégie combine plusieurs niveaux de tests.

5.3.1 Tests unitaires / de développement

- Objectif : valider de petites fonctions/scripts (parsing, transformation d'une colonne, validation de date).
- Méthode : exécuter les fonctions localement ou via pytest sur des extraits réduits.
- Artefacts : extraits de notebooks transformés en scripts testables (export `.py`).

5.3.2 Tests d'intégration

- Objectif : valider l'enchaînement des étapes : ingestion → Bronze → Silver.
- Méthode : effectuer un run ADF complet sur un jeu d'essai (dataset réduit), vérifier les sorties Silver attendues.
- Vérifications : schémas, comptages approximatifs, logs d'exécution.

5.3.3 Tests end-to-end (E2E)

- Objectif : valider le flux complet jusqu'à la publication Gold et l'actualisation Power BI.
- Méthode : exécution de `MasterIngestioDaily` + `BronzeToSilver` + `MasterSilverToGold`; rafraîchissement du dataset Power BI.
- Points de contrôle : `ingestion_status` mis à jour, `quality_gate_status` = PASS, tables Gold disponibles.

5.3.4 Tests de non-régression

- Objectif : s'assurer qu'une évolution du code ne casse pas les transformations.
- Méthode : exécuter une suite de tests E2E sur un jeu représentatif stocké comme « golden dataset ».

5.4 Plan de tests et scénarios

Exemples de scénarios à automatiser / exécuter manuellement :

1. Scénario A — Ingestion nominale :

- Générer 1 jour de données (fichiers CSV).
- Uploader via `AutomaticUpload.py`.
- Lancer `MasterIngestioDaily`.
- Vérifier que le fichier Bronze existe, que Silver est écrit, et que `quality_gate` = PASS.

2. Scénario B — Fichier corrompu / format incorrect :

- Injecter un CSV avec une colonne manquante.
- Lancer le pipeline; vérifier que l'activité signale l'erreur et que le fichier est isolé dans `/raw/_quarantine`.
- Vérifier que `quality_gate` FAIL empêche la promotion.

3. Scénario C — Doublons & déduplication :

- Générer deux snapshots contenant les mêmes `po_id` mais avec des `ingestion_ts` différents.
- Vérifier que la logique de déduplication conserve la ligne la plus récente.

5.5 Data Quality (DQ) — règles & automatisation

Rappel des règles DQ (implémentées dans `DataQuality_Checks.ipynb`) :

- Schéma présent (colonnes attendues).
- Clés naturelles non nulles (supplier_id, po_id, product_id).
- Unicité sur les identifiants primaires dans la fenêtre d'ingestion.
- Plages valides pour les dates et les quantités.
- Seuil de rejet (ex : 5%) pour la décision automatique.

Les résultats sont écrits dans /medallion/control/quality_gate_status.csv et exposés à Synapse afin d'être consommés par Power BI pour le monitoring.



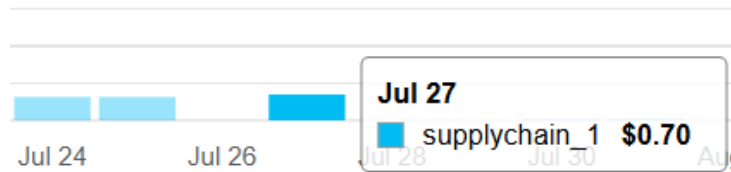
Exemple de dashboard Data Quality (image fournie dans le dépôt)

5.6 Métriques à collecter

Les métriques clés :

- Volume ingéré (octets/nombre de fichiers) par entité et par date.
- Temps d'ingestion (durées des Copy Activity ADF).
- Temps de traitement Spark (par notebook, driver + exécutants).
- Latence totale du batch (ingestion → Gold prêt).
- Taux d'erreur DQ (percent_rejected) et nombre de fichiers en quarantaine.
- Coût estimé par run (estimation stockage + calcul Synapse + ADF).

Un exemple d'illustration (consommation / coût) est disponible dans le dépôt : images/CostDaily.pr



Coût journalier — Petit jeu de données (image du dépôt)

5.7 Résultats de tests manuels (exemples)

Note : les valeurs ci-dessous sont des exemples de validation pédagogique (exécution sur environnement de formation). Pour la reproductibilité, conserver les logs ADF/Synapse et la valeur des paramètres (taille dataset, configuration du pool).

- Jeu d'essai (<1 Go) : latence end-to-end $\approx$ 15–30 minutes (selon le dimensionnement du pool Spark et les I/O).
- Quality Gates : en exécution standard, `percent_rejected` typiquement $< 2\%$ sur des datasets générés correctement ; fichiers corrompus détectés et isolés.
- Déduplication : les tests de concordance montrent la conservation de la ligne la plus récente pour les clés naturelles dupliquées.

5.8 Monitoring et alerting en production

Configuration recommandée :

- Centraliser les logs ADF & Synapse dans Log Analytics.
- Définir des règles d'alerte :
 - Pipeline en échec (ADF) $\rightarrow$ notification Teams / Email.
 - Durée de job $>$ seuil $\rightarrow$ investigation.
- Dashboards opérationnels (Power BI) :
 - Ingestions quotidiennes.
 - Métriques de qualité (pass / fail par entité).

Exemple visuel d'alerte/dashboards présenté dans le dépôt (images/).

5.9 Plan de validation & tests d'acceptation (UAT)

Critères d'acceptation pour la remise du pipeline :

1. Tous les pipelines ADF s'exécutent sans erreur sur le jeu d'essai (nominal).

2. Quality Gate PASS pour au moins 95% des entités sur le jeu d'essai (ou selon le seuil convenu).
3. Les tables Gold respectent le format et le schéma attendus (validé via requêtes de contrôle).
4. La documentation présente la procédure de relance et de dépannage.
5. Les tests de montée en charge montrent une évolutivité raisonnable (temps de run acceptable ou réduction réelle via scaling).

5.10 Automatisation des tests

Proposition d'automatisation (amélioration future) :

- Créer un pipeline CI qui, à chaque commit sur la branche main :
 - Exporte / versionne les notebooks (en .py).
 - Lance des tests unitaires locaux (pytest) sur les fonctions utilitaires.
 - Déploie en environnement de test (via ARM/Bicep/Terraform).
 - Lance un run E2E sur un dataset réduit (smoke test).
- En cas d'échec, bloquer la promotion vers la production et notifier l'équipe.

5.11 Recommandations post-tests

- Automatiser la collecte des métriques et exporter régulièrement les résultats DQ pour le suivi.
- Planifier des revues mensuelles des dashboards de qualité et de coût.
- Ajuster le pooling Spark et le partitionnement selon les patterns d'usage réels.
- Mettre en place des sauvegardes des tables Gold critiques et des procédures de recovery.

5.12 Conclusion

Les tests et la validation constituent une étape essentielle pour garantir la fiabilité de la plateforme. La combinaison de tests unitaires, d'intégration, E2E et de monitoring permet d'assurer une détection précoce des anomalies et la maîtrise des performances. Après validation, la promotion vers une stratégie CI/CD et un plan d'exploitation complet (Partie 6) permettra la mise en production et l'évolution contrôlée du pipeline.

Partie 6

Déploiement, Exploitation, Gouvernance et Conclusion

Déploiement, opérations et finalisation

6.1 Introduction

Cette partie finalise le rapport en présentant le plan de déploiement opérationnel, les activités d'exploitation (runbook), les aspects de gouvernance et de sécurité, le planning et les livrables attendus conformément au Cahier des Charges fourni par Smartovate. La section conclut sur les limites, les risques et les perspectives.

6.2 Rappels du Cahier des Charges

Rappel synthétique des objectifs du projet (durée 2 mois) :

1. Ingestion : pipelines robustes ADF vers ADLS Gen2.
2. Transformation & modélisation : Synapse (Spark et SQL) — Bronze / Silver / Gold.
3. Visualisation : Power BI (modèle sémantique et rapports).
4. Automatisation & Monitoring : actualisation planifiée et alertes qualité.

Périmètre (In) : configuration Azure, pipelines ADF, stockage ADLS Gen2 (Medallion), transformations Synapse, rapports Power BI, alertes qualité.

Hors périmètre (Out) : algorithmes ML complexes, migration de volumes historiques massifs, intégration on-premise complexe.

6.3 Plan de déploiement (sans CI/CD)

Ce paragraphe décrit comment déployer et configurer les composants manuellement / semi-automatiquement en suivant le Cahier des Charges, sans intégrer d'outil CI/CD dans le périmètre du projet.

6.3.1 Étapes de provisionnement et configuration

1. Provisionner les ressources Azure (resource group) :

- Storage Account (ADLS Gen2) : créer les conteneurs source/raw, bronze, silver, gold, control.
- Azure Data Factory : créer l'instance et les linked services (ADLS, Synapse).
- Azure Synapse Workspace : créer le workspace Synapse, le Spark pool et le SQL pool/Serverless.
- Azure Key Vault : créer et ajouter les secrets nécessaires.
- Power BI : préparer le workspace de publication (Power BI Service).

2. Initialiser la structure du Data Lake :

- Créer les répertoires Bronze/Silver/Gold et appliquer les ACLs sur ADLS (Managed Identities).
- Déployer les fichiers de configuration et la control table (`ingestion_status.csv`) dans `/medallion/control/`.

3. Déployer les artefacts ADF (manuellement via l'UI ou en important des templates ARM fournis) :

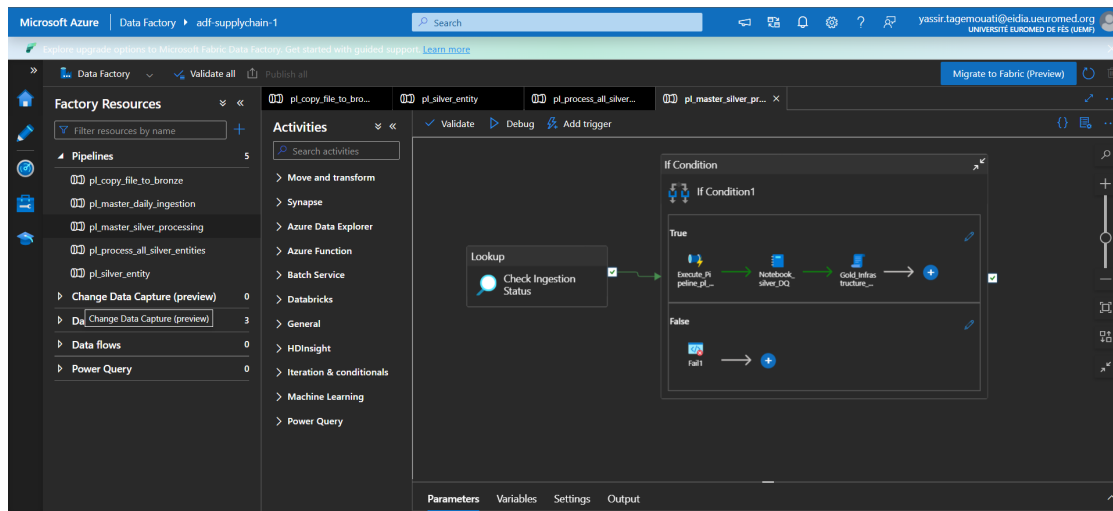
- Importer les pipelines : `MasterIngestionDaily`, `BronzeToSilver`, `MasterSilverToGold`.
- Configurer les linked services, datasets et triggers (scheduling quotidien).

4. Déployer les artefacts Synapse :

- Importer les notebooks PySpark (Bronze → Silver et contrôles DQ).
- Importer les scripts SQL (`GoldShema.sql`, upserts, vues).
- Tester l'exécution des notebooks depuis Synapse Studio.

5. Publier les rapports Power BI (manuellement) :

- Connecter Power BI Desktop à Synapse (DirectQuery/Import selon le choix).
- Développer et tester les rapports localement, puis les publier dans Power BI Service.



Pipeline de promotion Silver → Gold déployé dans ADF (image du dépôt)

6.3.2 Configuration des accès et sécurité

- Activer la Managed Identity pour ADF et Synapse ; accorder les ACLs lecture/écriture sur ADLS.
- Stocker les secrets nécessaires (SAS, service principal, credentials éventuels) dans Key Vault.
- Appliquer un RBAC minimal sur les ressources (moindre privilège).
- Documenter toutes les permissions et leurs propriétaires pour audit.

6.4 Exploitation et runbook opérationnel

Le runbook décrit les opérations quotidiennes et les procédures d'intervention.

6.4.1 Opérations quotidiennes

1. Vérifier l'exécution des pipelines ADF (monitor) ; corriger les erreurs ou relancer si besoin.
2. Contrôler les métriques principales : volumes ingérés, temps d'exécution, erreurs.
3. Consulter `quality_gate_status` (dans `/medallion/control/`) et le dashboard Data Quality.
4. S'assurer que les actualisations Power BI se sont terminées (si planifiées).

6.4.2 Procédure en cas d'incident

1. Identifier l'étape en échec via ADF Monitor / logs Synapse.
2. Extraire les logs pertinents et la raison de l'erreur.
3. Isoler le fichier source problématique dans `/raw/_quarantine`.
4. Si erreur de schéma, corriger le fichier localement ou ajuster le script de transformation si une tolérance est prévue.
5. Mettre à jour `ingestion_status.csv` (status = Pending/Processed) si une relance est nécessaire.
6. Relancer le pipeline ADF manuellement pour la partition/entité concernée.
7. Documenter l'incident (cause, correction, actions préventives).

6.4.3 Tâches hebdomadaires / mensuelles

- Revue des dashboards Data Quality et des KPI d'ingestion.
- Purge / archivage des données brutes plus anciennes selon la politique (ex : archivage > 90 jours).
- Réconciliation des volumes Silver vs Bronze (contrôles ponctuels).
- Vérification des coûts et ajustement des ressources (dimensionnement du Spark pool).

6.5 Monitoring et alerting

Conformément au Cahier des Charges (Automatisation et Monitoring) :

- Configurer Azure Monitor / Log Analytics pour centraliser les logs ADF et Synapse.
- Définir des alertes :
 - Échec d'un pipeline ADF → notification email/Teams.
 - Quality gate FAIL → alerte opérationnelle.
 - Dépassement de la durée d'exécution (seuil) → investigation.
- Construire un tableau de bord Power BI pour le suivi opérationnel : volumes ingérés, temps, métriques qualité, erreurs.

6.6 Backlog fonctionnel et critères d'acceptation

Le Cahier Fonctionnel (backlog / User Stories) fourni est repris ici et sert de base pour la validation.

6.6.1 Epic 1 : Infrastructure et Sécurité

- **US 1.1 — Provisionnement** : ressources créées, dossiers ADLS initiaux, Managed Identities activées.
- **US 1.2 — Key Vault** : secrets stockés, accès ADF/Synapse via Managed Identity.

6.6.2 Epic 2 : Ingestion (ADF)

- **US 2.1 — Pipeline Bronze** : Copy Data, partitionnement par date, gestion des erreurs.
- **US 2.2 — Paramétrage** : pipelines dynamiques, control table pour le pilotage.

6.6.3 Epic 3 : Transformation & Modélisation (Synapse)

- **US 3.1 — Silver** : nettoyage, doublons supprimés, conversion en Parquet/Delta.
- **US 3.2 — Gold** : schéma en étoile, clés, optimisation du SQL pool.

6.6.4 Epic 4 : Power BI

- **US 4.1 — Modèle sémantique** : relations correctes, mesures DAX.
- **US 4.2 — Dashboard exécutif** : pages (Overview, Details, Temporal Analysis) et conformité graphique.

6.6.5 Epic 5 : Monitoring & Quality

- **US 5.1 — Alertes DQ** : requêtes quotidiennes de contrôle et alertes.
- **US 5.2 — Monitoring ADF** : alertes en cas d'échec de pipelines.

Chaque User Story doit être validée par les critères d'acceptation décrits dans le cahier fonctionnel.

6.7 Gestion des bugs potentiels et actions préventives

Extraits des bugs identifiés dans le Cahier des Charges, avec les actions recommandées :

TABLE 6.1 – Bugs potentiels et actions préventives

Bug	Action préventive
Échec d'authentification Power BI ↔ Synapse	Préférer Managed Identity / Service Principal avec permissions durables ; vérifier l'expiration et les renouvellements ; documenter la configuration dans Key Vault.
Timeouts Direct-Query / Performance	Optimiser les modèles Synapse (distribution, index columnstore), limiter les visualisations lourdes par page, planifier des agrégations (vues matérialisées) ou utiliser Import pour les datasets volumineux.
Corruption d'écriture concurrente	Adopter Delta Lake pour la gestion ACID si requis, ou contrôler le séquençage des écritures via l'orchestration ADF (locks, dépendances), éviter la concurrence d'écriture sur une même partition.

6.8 Planning révisé et réalisation (alignement Cahier des Charges)

Durée du projet : 2 mois (8 semaines) — planning conforme au cahier :

- Semaine 1-2 : Cadrage & Setup Azure (provisionnement, IAM, structure ADLS).
- Semaine 3-4 : Développement des pipelines d'ingestion (ADF) et premiers nettoyages (Bronze → Silver).

- Semaine 5-6 : Transformations avancées et modélisation Gold (Synapse SQL / Spark).
- Semaine 7 : Développement des rapports Power BI et paramétrage des actualisations.
- Semaine 8 : Tests E2E, recette, documentation et présentation finale.

6.9 Limites du projet et recommandations finales

- Données utilisées : jeux simulés Oracle SCM créés par l'étudiant — à revalider avec des sources réelles en cas de migration future.
- Intégrations on-premises / réseaux complexes : hors périmètre ; prévoir des études réseau et sécurité pour une intégration en production.
- Machine Learning et traitements temps réel : hors périmètre mais listés comme axes d'évolution.

Recommandations :

- Valider la solution sur un jeu de données réel avant industrialisation.
- En cas de passage en production : renforcer la gouvernance (Data Catalog / Purview), les sauvegardes et politiques de rétention, ainsi que les procédures d'audit.
- Envisager Delta Lake si un besoin d'ACID et d'upserts à grande échelle se présente.

Webographie

Voici une sélection organisée de ressources techniques, de documentations officielles et d'outils en ligne utilisés pour la conception et l'implémentation de la plateforme Azure présentée dans ce rapport.

Estimation des coûts

- **Azure Pricing Calculator** — calculateur officiel des coûts Azure
<https://azure.microsoft.com/pricing/calculator>
- **AWS Pricing Calculator** — calculateur officiel des coûts AWS
<https://calculator.aws/>
- **Comparatif AWS ↔ Azure** — cartographie et équivalences de services (Microsoft Learn)
<https://learn.microsoft.com/azure/architecture/aws-professional/>

Ingestion et orchestration

- **Azure Data Factory** — documentation et tutoriels (orchestration, pipelines, Copy Activity, Data Flows)
<https://learn.microsoft.com/azure/data-factory/overview>
- **Parcours Microsoft Learn** — ingestion et transformation de données avec ADF
<https://learn.microsoft.com/training/paths/ingest-transform-load-data-azure-data-factory/>

Traitement et stockage

- **Azure Synapse Analytics** — documentation (Synapse Studio, Spark pools, SQL Serverless & Dedicated)
<https://learn.microsoft.com/azure/synapse-analytics/>
- **Azure Data Lake Storage Gen2** — concepts et bonnes pratiques
<https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-introduction>

- **Apache Parquet** — format de stockage columnar
<https://parquet.apache.org/documentation/latest/>
- **Delta Lake** — transactions ACID pour Data Lakes
<https://delta.io/>
- **PySpark** — API Python d'Apache Spark
<https://spark.apache.org/docs/latest/api/python/>

Sécurité et gouvernance

- **Azure Key Vault** — gestion des secrets et Managed Identity
<https://learn.microsoft.com/azure/key-vault/general/overview>
- **Sécurité Azure** — principes et bonnes pratiques (Identity, RBAC, chiffrement, gouvernance)
<https://learn.microsoft.com/azure/security/fundamentals/>

Monitoring et supervision

- **Azure Monitor** — centralisation des logs, métriques et alertes
<https://learn.microsoft.com/azure/azure-monitor/overview>
- **Log Analytics** — guide détaillé
<https://learn.microsoft.com/azure/azure-monitor/logs/log-analytics-overview>

Restitution et modélisation

- **Power BI** — guides Desktop, Service, modélisation et rafraîchissement
<https://learn.microsoft.com/power-bi/>
- **Power BI** — conception de modèles performants (DirectQuery, Import, modèles composites)
<https://learn.microsoft.com/power-bi/guidance/>

Architecture et référentiels

- **Medallion / Lakehouse Architecture** — guide conceptuel Databricks (Bronze / Silver / Gold)
<https://databricks.com/glossary/medallion-architecture>
- **Azure Architecture Center** — bonnes pratiques architecturales et patterns
<https://learn.microsoft.com/azure/architecture/>

Ressources communautaires

- **Azure Samples & Quickstarts**
<https://learn.microsoft.com/azure/samples/>
- **Microsoft Tech Community** — blog Azure
<https://techcommunity.microsoft.com/>

Conclusion Générale

Le projet, tel que défini par le Cahier des Charges de Smartovate, a permis de concevoir et d'implémenter une chaîne de traitement de données complète et pédagogique centrée sur Azure : ingestion via ADF, transformations Synapse (Spark/SQL), stockage ADLS Gen2 (Medallion) et restitution Power BI.

Les livrables et la documentation fournis permettent la reprise du projet par une équipe Data pour exploitation ou extension, en tenant compte des recommandations de sécurité et de gouvernance présentées tout au long de ce rapport.

Ce travail constitue la synthèse technique du stage réalisé au sein de Smartovate Ltd, sous la direction académique de Mme Loubna Ourabah (EIDIA, UEMF) et l'encadrement professionnel de M. Abdelkhalek Bakkari, Fondateur et CEO de Smartovate. Il pose également les bases pour de futures industrialisations ou évolutions fonctionnelles de la plateforme.